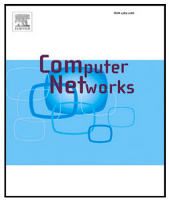
Computer Networks 245 (2024) 110398



Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet





Phishing URL detection generalisation using Unsupervised Domain Adaptation

Fariza Rashid [a,*], Ben Doyle [b], Soyeon Caren Han [c], Suranga Seneviratne [a]

[a] School of Computer Science, The University of Sydney, Australia
[b] Thales, Australia
[c] The University of Western Australia, The University of Sydney, Australia

ARTICLE INFO

Keywords:
Phishing detection
Unsupervised domain adaptation
URL classification

ABSTRACT

Phishing attacks are a prevailing problem in cybersecurity. In many data breaches, the initial entry can be traced back to phishing. URL-based phishing detection is one of the many ways of phishing attempt detection where solely the properties of the URLs are used to decide whether a given URL is phishing or not. While there are multiple existing works that use machine learning and deep learning to detect phishing URLs, in this paper, we show that such methods lack generalisation (i.e., they work effectively only when the test sets are split from the same training dataset). This is a significant issue since the vast majority of phishing attempts are short-lived and use freshly created domain names. Also, many network vantage points and middleboxes record URLs in slightly different formats and as such, URL data collected at various companies may be different. To address this, we propose an Unsupervised Domain Adaptation-based framework to increase the model transferability between datasets. We evaluate our approach using three datasets and show that the increase in cross-dataset F1 score performance is 0.06 on average and in some cases approximately as high as 0.2.

1. Introduction

A significant proportion of cyber attacks on the internet comprise phishing attempts to harvest sensitive information such as user credentials. Recent reports estimate that phishing emails are the entry points of 91% of all cyber attacks [1]. As such, identifying phishing URLs accurately and efficiently is essential when it comes to securing corporate networks.

The most prevalent approach to detect phishing attempts includes maintaining blacklists of previously identified phishing URLs and giving a cautionary warning or preventing user access to those URLs, such as Google's Safe Browsing [2] and Microsoft Smartscreen [3]. Nonetheless, these methods can be inefficient. For example, blacklists can take close to an hour to list unsophisticated phishing websites and close to three hours to list phishing websites employing minor evasion techniques such as redirection via URL shorteners [4]. They can also fail to identify phishing attacks employing sophisticated evasion techniques [4]. Many phishing attempts capitalise on the time gap for blacklists to identify a phishing URL to conduct short-lived phishing campaigns.

To overcome the limitations of basic blacklist-based phishing URL detection, machine learning-based detectors have been proposed [5,6], which conventionally rely on the features collected from the phishing web page (e.g., page title, images, HTML information). However, these detectors also can be bypassed by more advanced evasion techniques such as *cloaking* where phishing content on a webpage is replaced with benign content when the page is being accessed by a phishing detection crawler [7]. Other more advanced evasion techniques include redirection via shortened URLs and hosting phishing content on compromised legitimate websites. Overcoming these evasion techniques requires specific attention to the finally visited URL itself. This has paved the way to *URL-based phishing detectors* that solely use the features of the URL to predict whether a given URL is a phishing attempt or not [8,9]. The advantage of these methods is that they do not require visiting the corresponding web page in the background to make the prediction as the features of the target web page are not used. Such solutions have used machine learning methods [8,10,11] as well as more recently deep learning methods [12,13].

Nonetheless, these state-of-the-art models have always been tested only on one dataset where the training and test sets have the same distribution. As we show in our later analysis, their performance is poor across datasets and, as such, they lack generalisability, creating interoperability issues between various networks as well as being vulnerable

* Corresponding author.
E-mail addresses: fariza.rashid@sydney.edu.au (F. Rashid), ben.doyle@thalesgroup.com.au (B. Doyle), caren.han@uwa.edu.au (S.C. Han), suranga.seneviratne@sydney.edu.au (S. Seneviratne).

https://doi.org/10.1016/j.comnet.2024.110398
Received 20 October 2023; Received in revised form 7 February 2024; Accepted 3 April 2024
Available online 6 April 2024
1389-1286/© 2024 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

to new phishing URLs that are perpetually generated to outmanoeuvre classifiers. To overcome this limitation, we conduct statistical tests across multiple URL datasets and identify the features with distribution shifts causing the models to perform poorly. We identify the issue to be a domain adaptation problem, as the distribution shifts are introduced as a result of collecting URLs from different sources. To mitigate this issue, we propose a feature alignment framework based on Unsupervised Domain Adaptation. More specifically, we make the following contributions.

- We evaluate the performance of several state-of-the-art phishing URL detectors across multiple datasets and show that their performance can drop as much as 10%–32% when the test set is coming from a different dataset than the original training set.
- Through statistical tests, we identify the features that are causing the performance drop and the nature of their distributional shifts. For example, we found that distributional shifts in URL length and the classifiers' focus on the presence of some specific words (e.g., 'www') contribute to performance drops, while counterintuitively, the appearance of new words unseen during training does not affect transfer performance for certain models.
- Based on these findings, we propose an Unsupervised Domain Adaptation (UDA) approach for URL classifiers when training and test sets are not from the same dataset. To the best of our knowledge, this is the first work to propose domain adaptation in the context of URL classifiers.
- We evaluate the performance of our approach using four existing methods and three datasets and demonstrate an increase in F1 score by 0.06 on average and approximately up to 0.2 in some scenarios.

The rest of the paper is organised as follows. In Section 2 we describe related work in phishing detection and also introduce the field of Unsupervised Domain Adaptation. Section 3 describes how our framework aligns with Unsupervised Domain Adaptation. Next, using three different phishing datasets, we conduct our analysis of four existing phishing detection models. In Section 4 we introduce the datasets we use and in Section 5 we show that state-of-the-art phishing detection models have low transfer accuracies when evaluated using datasets other than the one they were trained on. In Section 6 we analyse distributional differences of various features between datasets using statistical tests and identify the features that cause a performance drop in transfer performance. Following this we introduce our feature alignment approach in Section 7 and evaluate its performance in Section 8. Finally, in Section 9, we discuss the implications and limitations of our work and also make recommendations for future work before concluding the paper.

2. Related work

2.1. Approaches to phishing detection

Multiple works proposed machine learning-based phishing URL detection methods. These models extract features from the URLs, landing pages, and hosting information and train machine learning classifiers to predict whether a given URL is phishing or benign. Shahoo et al. [14] broadly categorised these features as blacklist features, lexical features, host-based features, and content-based features. Blacklist features denote the presence of a URL or a slightly modified version of it in a blacklist [15,16]. Works such as [15,17–19] implemented URL lexical features by identifying the presence of words delimited by special characters as well as host-based features such as domain name properties and IP address properties. Several works have also used content-based features such as HTML and Javascript information [5,6,20–22], as well as textual and visual contents [23–25]. Others further incorporated web-page hyperlinks information [26,27] such as calculating the ratio of the most common anchor link to the total number of links on the page. In contrast to these, Tan et al. [28] proposed graph-based methods by leveraging web-page hyperlinks and page-linking data. Finally, several works also used additional features, such as domain age information from WHOIS or page rank from the Alexa top domains list, derived from third-party sources [20,22,29–31].

2.2. URL-based phishing detection

Most of the above-described features other than the URL features, are difficult to obtain in real-time and such methods can be vulnerable to evasion techniques [4]. Also, generating content features requires visiting the phishing web page in the background. This poses risks of accidental malware download and retaliatory actions, even though it is done in a machine-to-machine manner. Moreover, obtaining hosting information is another difficulty for phishing websites, as they are often short-lived. To address these limitations, URL feature-based methods have been proposed as a complementary alternative. These methods do not rely on features from third-party sources, pose no risk of accidental download of malware, and are not susceptible to webpage-based evasion techniques such as cloaking. Furthermore, these methods can attend to the issue of phishing sites hosted on compromised legitimate websites as they focus only on the URLs. In this paper, we particularly focus on this specific type of phishing detection method that has been explored in multiple other previous work [11,13,32]. Also, it is worth noting that URL-based phishing detection can also be combined with other content-based methods as well. Overall, URL-based phishing detection methods can be broadly categorised into two.

2.2.1. Using hand-crafted features

These methods involve extracting hand-crafted features from the URLs such as URL length, word counts, presence of special characters and subdomain counts, and using those features to train machine learning models such as Random Forests and Support Vector Machines [8,10,11]. Examples of such methods include CatchPhish [11] and Sahingoz et al. [10], which demonstrated accuracies of 98.25% and 97.98%, respectively. However, both works evaluated their models only on the datasets they collected and introduced in the paper.

2.2.2. Using embedding representations

With the advances in deep learning methods, several works proposed deep-learning-based models or deep learning-based representations for phishing URL detection. For instance, Yuan et al. [33] proposed to convert each URL to component-wise character embeddings learned using a SkipGram language model [34] and trained machine learning classifiers such as XGBoost, Logistic Regression and Random Forest using these embedding representations. On the other hand, Rao et al. [12] fed URL word embeddings into an LSTM architecture, from which a feature layer was extracted to build an ensemble of SVMs.

Other works have implemented end-to-end deep learning pipelines. Both [13,35] are similar in that an embedding representation of the URL is used to train a CNN classifier. URLNet [13] first randomly initialises and trains embeddings for each word in an input URL delimited by special characters. Then, it computes embeddings for characters within each word, which are combined with the word-level embeddings by element-wise addition. These embeddings, along with character-level embeddings, are then passed through a CNN architecture for the final prediction. In a similar work, Aljofey et al. [35] also trained a CNN classifier for phishing detection, however, using only character-level embeddings. URLNet reported a true positive rate of 95.58% at a false positive rate of 0.1, while Aljofey et al. [35] reported an F1 score of 95.13%.

More recently, Maneriker et al. [32] proposed URLTran, a model which leveraged state-of-the-art transformer models for phishing detection. Unlike previous approaches, this work uses sub-word tokenised

URLs to fine-tune a BERT model [36] that was pre-trained on a massive corpus of natural language documents, and it is considered that this step learns contextual information among URL tokens. The URL tokens are then processed through this model, and the output embeddings are used to train a feed-forward network that generates a phishing prediction. In comparison to URLNet, Maneriker et al. report that URLTran returns a true positive rate (TPR) relative improvement of 21.9% at the false positive rate (FPR) of 0.01%.

In contrast to these works, we do not propose a new classification model for phishing URL detection. Instead, we investigate how existing models perform across datasets and show that their performances are limited beyond the test sets that are split from the same training dataset. To address this common issue, we propose and evaluate a feature alignment framework based on Unsupervised Domain Adaptation.

2.3. Unsupervised domain adaptation

Domain adaptation belongs to the larger family of transfer learning, which aims to train a model to perform well on data from a target domain based on knowledge obtained from a different source domain. Within transfer learning, domain adaptation belongs to the subcategory known as transductive transfer learning, in which the domain changes between the source and the target while the task remains the same. An example from computer vision is when a model trained on images of handwritten digits, such as the MNIST dataset, is used to detect house numbers in images such as the SVHN dataset [37]. Both the source and target data consist of images of digits; however, differences arising from the sourcing of the data introduce a domain shift between the two datasets.

Therefore applications of domain adaptation include computer vision [38,39], natural language processing [40,41], speech recognition [42,43], cross-domain recommendation systems [44] and activity recognition [45,46]. For instance, Khan et al. [45] proposed a domain adaptation solution to address the performance degradation in human activity recognition deep learning models between different devices. Models trained using human activity sensor data (e.g., running and walking) collected using a specific device such as a smartwatch are found to have a performance drop when used to identify human activity from data of the same type but collected from a different device such as a smartphone.

A subcategory of domain adaptation is Unsupervised Domain Adaptation (UDA), which assumes that the source domain is labelled while the target domain is unlabelled. UDA aims to reduce the need for labelled target domain data, as this is often expensive to obtain. Examples of work in this area include [47,48]. In both studies, a model trained on images captured using a DSLR, for example, is used to recognise the same objects in images captured using a webcam. The different image sources are considered to be different domains, and the target domain (webcam images) is assumed to be unlabelled. Other work, such as [49], used Unsupervised Domain Adaptation for speaker verification systems where a model trained on labelled speech data in the English language is used for speaker verification on unlabelled Chinese language speech data.

UDA is particularly applicable to the phishing detection problem as phishing URLs are short-lived, and models trained on a particular set of URLs (source domain) need to be effective in detecting newly generated phishing URLs (target domain) in addition to the usual challenges in costs associated with obtaining labelled data. To the best of our knowledge, this is the first work to propose a UDA-based solution to address the cross-dataset portability challenges of phishing URL detection models.

3. Our framework

In this section, we introduce how we apply Unsupervised Domain Adaptation (UDA) in phishing URL detection. First, we explain how we adapt the general theoretical UDA framework, and then we provide specific details of our approach.

3.1. UDA theoretical framework

An outline of the general UDA theoretical framework can be found in [50] where the domain D is defined as $D = \{X, Y, p(x, y)\}$. For our phishing detection problem, each dataset represents a different domain. Therefore $X \subset \mathbb{R}^d$ is a subset of a d-dimensional space representing the URL input domain of a unique dataset, and $X = x_1, x_2, \ldots, x_n$ where n is the number of samples and each x_i is a d-dimensional representation of a URL sample. For classifiers that use hand-crafted features [10,11], d represents the total number of hand-crafted features — examples of which include the URL length or the word count of a URL. For models like [13,32] that use embedding representations, d is the total number of tokens considered as input from each URL sample; that is, the sequence length.

As phishing detection is a binary problem, the label space $Y = \{0, 1\}$, and $p(x, y)$ represents the joint probability distribution over X and Y. As stated in [50], in Unsupervised Domain Adaptation, a function f is learnt that maps the input to the label space, $f : X \rightarrow Y$, by minimising the expected risk:

$$R_T(h) = \mathbb{E}_{(x,y)\ P_s}[\frac{P_T(x)}{P_S(x)} l(f(x), y)] \tag{1}$$

where P_s is the source distribution, P_T refers to the target distribution, $l(f(x), y)$ is a loss function denoting the difference between the model prediction and the original label, and $R_T(h)$ is the sum of all the disagreements in predictions. Here, the ratio of the two marginal distributions is referred to as the importance weight, $w(x) = \frac{P_T(x)}{P_S(x)}$, and is used to reweight the source domain data such that it is made similar to the target distribution.

This process of minimising the discrepancy between the two domains is addressed through various methods of UDA [51–54]. Most of the existing approaches are mainly deep-learning based. For example, [55] proposes a complete deep feed-forward network that combines three processes – the feature extractor, label predictor, and domain classifier – for domain adaptation. However, this approach can only be applied to networks that are trained using backpropagation. Secondly, these approaches consider the entire input feature space $x \in \mathbb{R}^d$. That is, they aim to reduce the distance between the latent feature representations of the source and the target domains and, consequently, fail to reveal the specific low-level features in the original data samples that contribute to domain shift. This is often because the application of these works is in computer vision where extracting meaningful low-level features is difficult.

In contrast, we implement UDA in a more generic way such that it can be applied to both machine learning and deep learning models (cf. Section 3.2). Furthermore, we investigate individual features for domain adaptation instead of the entire feature space as a whole. By focusing on specific features with different distributions between the source and target domains, we aim to uncover the causes of overfitting that are specific to different datasets as well as the causes that are common across all of them.

Therefore, for our approach, Eq. (1) changes to

$$R_T(h) = \mathbb{E}_{(x,y)\ P_s}[\frac{P_T(x')}{P_S(x')} l(f(x), y)] \tag{2}$$

where $x' \in \mathbb{R}^{d'}$ are the URL samples represented as a subset of the original d-dimensional feature space, and $d' \leq d$ is the number of features that we expect to have different distributions between the source and target domains. Therefore, we restrict our reweighting of the source domain $(w(x))$ to only those features that have different distributions between the source and target domains.

3.2. Phishing URL detection using UDA

In Fig. 1 we summarise our overall UDA approach for phishing URL classification applicable for both machine learning and deep learning models.

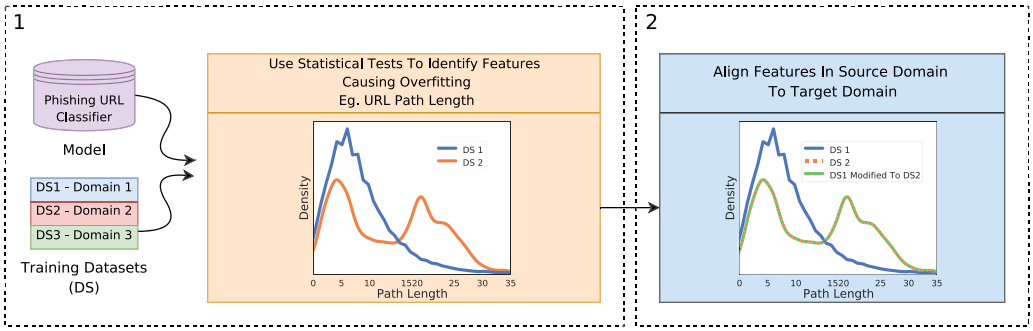


Fig. 1. Phishing URL detection using UDA.

1. For each model we first uncover the d' exact features that hinder transfer performance between different phishing URL datasets. To do that we use statistical tests to determine if there is a shift between the source and target domains with respect to these features. Further details of this process and the features we identified as overfitting are stated in Section 6.
2. Next, we align the features in the source domain to the target domain. We propose different alignment procedures as appropriate for different features and explain this process in further detail in Section 7.

4. Datasets

We use two publicly available URL datasets and one URL dataset compiled by us in this paper.

4.1. ISCX-2016 dataset

The ISCX-2016 dataset was collected by Mamun et al. [8]. It consists of benign URLs and various malicious URL types, including malware, spam, defacement, and phishing. Although the dataset contains more than 100,000 URLs altogether, there are only 9964 phishing and more than 35,300 benign URLs. Thus, to obtain a more balanced dataset, we randomly selected 10,000 benign URLs and combined them with the phishing URLs to build the ISCX dataset for this paper. The phishing URLs in this dataset were sourced from OpenPhish,[1] and the benign URLs were collected by crawling Alexa top domains.

4.2. EBBU-2017 dataset

The EBBU-2017 dataset was introduced in [10], and it comprises 36,400 benign URLs and 37,175 phishing URLs. The phishing URLs were collected from PhishTank,[2] while the benign ones were collected using the search engine Yandex's search API.[3] The authors constructed a specific list of words with which they queried the Yandex search engine. The highest-ranked URLs that were returned from this query were collected as benign URLs under the assumption that phishing URLs would not be given a high ranking by search engines.

4.3. HISPAR-Phishstats dataset

Since ISCX-2016 and EBBU-2017 are somewhat older, we created a new URL dataset consisting of benign and phishing URLs obtained from the HISPAR list [56] and Phishstats,[4] respectively. The HISPAR list contains the landing and internal page URLs for around 2000 top domains that appear on the Alexa top domains list. We collected our benign URLs from this list by extracting an equal number of internal pages for each domain available. As the number of internal pages for each domain differed, we tried to maximise the number of internal pages that was possible to collect and finally obtained 25 URLs for each of 1850 unique domains to create a benign list of 46,250 URLs.

We collected phishing URLs from Phishstats, a website that frequently publishes lists of new phishing URLs. We collected the published lists between the 19th of June and the 26th of August 2022, filtered out all duplicate URLs, and randomly selected 46,250 URLs to ensure our resulting dataset was balanced. The regular feed from Phishstats contains a score denoting the likelihood of a URL being a phishing URL such that: 0–2 means likely, 2–4 means suspicious, 4–6 means phishing and 6–10 means definitely phishing. We selected only the URLs with a score of greater than five to ensure that the URLs in our dataset are indeed phishing. Other works that have used PhishStats as a source for phishing URLs [57–59] did not involve this additional filtering step.

We randomly split these datasets into training, validation and test sets in a 60:20:20 ratio as summarised in Table 1. We plan to release the new dataset upon the publication of this work to stem research in this area further.

Table 1
Dataset summary.

Dataset	Training		Validation		Test	
	Ben	Phish	Ben	Phish	Ben	Phish
ISCX	5,978	6,001	2,043	1,950	1,979	2,014
EBBU	21,804	22,341	7,227	7,488	7,369	7,346
HP	27,740	27,760	9,180	9,320	9,330	9,170

5. Transfer performance of baselines

Using four state-of-the-art models for phishing URL classification, we first show that their performances do not translate across datasets.

In our selection we prioritised models that used URL features only (cf. Section 2). Among the recent works that focus on URL features only, we selected two that use embedding features and two that use hand-crafted features. From models using embedding features, we selected *URLNet* [13] as this was the most highly cited among the available works. As a second model in this category, we selected *URLTran* [32] as it utilises transformer models that have demonstrated significant success in the area of text classification in general. From among the three models using hand-crafted features that we identified in Section 2.2.1, we selected the two most recent papers. These are *CatchPhish* [11], and the model by Sahingoz et al. [10] which we refer to as PhishRF. We give a summary of the models below.

[1] openphish.com
[2] phishtank.org
[3] yandex.com
[4] phishstats.info

Table 2
F1 scores for baseline experiments using the URLNet, URLTran, CatchPhish, and PhishRF models.

Train sets	Models	Test sets		
		ISCX	EBBU	HP
ISCX	URLNet	**0.99**	0.69	0.68
	URLTran	**0.99**	0.68	0.67
	CatchPhish	**0.99**	0.68	0.69
	PhishRF	**1**	0.68	0.67
EBBU	URLNet	0.88	**0.98**	0.92
	URLTran	0.83	**0.99**	0.93
	CatchPhish	0.80	**0.94**	0.88
	PhishRF	0.87	**0.96**	0.94
HP	URLNet	0.72	0.79	**0.99**
	URLTran	0.75	0.86	**0.99**
	CatchPhish	0.73	0.72	**0.98**
	PhishRF	0.83	0.86	**0.99**

5.1. URLNet [13]

URLNet is a CNN-based model comprising two CNN branches — one processes a character-level representation of the input URL while the other processes a word-level representation. The input to each CNN branch is an embedding representation of the characters and words, respectively, where two separate embedding matrices are learnt for all the unique characters and words in the training corpus. The words of the URL corpus are obtained by parsing the input URL by special characters, and the output of the CNN architectures from the two branches are concatenated and passed onto a fully connected network for classification.

5.2. URLTran [32]

URLTran model involves first fine-tuning a pre-trained BERT masked language model (MLM). This is done using the URL corpus, where each URL is tokenised using a pre-trained BERT subword tokeniser. The tokens are converted to embeddings, masked according to the original BERT masked language modelling specifications, and passed through the BERT model to predict the correct tokens. A dense layer with the number of classes matching the vocabulary size of the corpus is used for prediction.

After fine-tuning this model, the output of this model is used to train a classification network. Specifically, the CLS token output, which acts as a special pooled token representing the entire input sequence, is passed through a fully connected network that leads to a two-class classification layer.

5.3. CatchPhish [11]

CatchPhish uses manually extracted features from the URL as input features to train a random forest model. One type of such features includes count-based features. These features represent the number of possible phishing indicator tokens, such as the count of digits, dots, and hyphens. The other type of features involves the presence of tokens that are rare in benign URLs but frequent in phishing URLs. These features check the presence of either special characters or words likely to appear in phishing URLs such as the '$' or '*' characters. These features are generated for the hostname and the whole URL separately. This model also uses TF-IDF features of the training dataset vocabulary and, finally, checks for the presence of brand names that phishing attackers often target. The full list of features can be found in the original paper [11].

5.4. PhishRF [10]

This approach initially conducts data pre-processing on all URLs before extracting hand-crafted features, which are used to train a random forest model. The aim of the data pre-processing is mainly to parse the URL into words and identify similarities between the words and known brand names, and phishing keywords. Some of the features that are extracted thereafter are similar to those of CatchPhish, such as digit count, average word length, longest word length, and the presence of special characters. However, there are a few unique features, such as the count of words that are similar to brand names and phishing keywords and the count of words that contain a random sequence of characters. The full list of features can be found in the original paper [10].

For each model, we performed hyperparameter-tuning for each validation dataset and used the model with the best parameters to obtain predictions for each of the test sets. We provide a summary of these baseline results in Table 2 and use the F1 score as a performance metric as it provides a balanced reflection of the true positive and false positive rates. This is important in phishing detection, where high false positive rates are a common issue. Overall, our baseline results reflect that the F1 score is high when testing on the test splits from the same datasets and lower when testing on test splits from different datasets. We hypothesise that the reason for this involves differences in the distributions of the input features across the three datasets. As the input features to each detection model are different, in the next section we analyse the relevant features for each model separately.

6. Analysis of transfer performance

We next investigate the features among our three datasets that are likely to be overfitting features and potentially contribute to low transfer accuracy for each model, respectively. Our analysis here is two-fold. First, we check whether an existing distributional shift in the URL vocabulary (i.e., the presence of new words) is a source of the problem. This analysis is applicable for the URLTran and URLNet models that use word-level embedding representations and the CatchPhish model that uses TF-IDF values of the words in the vocabulary. We highlight that PhishRF does not use any vocabulary-based features. Therefore, this analysis does not apply to it. Second, for CatchPhish and PhishRF, we focus on the distributional differences of the most important hand-crafted features and check whether they cause the accuracy drop.

6.1. New words and contradictory words

We define two types of new words that can affect the cross-dataset performance: *new words* and *contradictory words*. Here, we highlight that the training and test sets are coming from two different datasets.

- **New words:** words that appear in the test set samples but not in the training set samples.
- **Contradictory words:** words with differing frequency patterns between training and test sets (e.g., words that are prevalent in benign samples in the training set but the opposite in the test set).

To obtain the contradictory words, we extract the set of common words for every training and test set pair, identify the majority label for each word by counting the number of benign and phishing URLs that they appear in, respectively, and select the label with the most counts. Then, we shortlist contradictory words by filtering the words with contradicting majority labels between the training and test sets.

For each training and test set pair, we compared the number of correctly and incorrectly classified URLs based on whether they contained a certain number of new words or contradictory words. We observed in general that for the URLNet and URLTran models, the

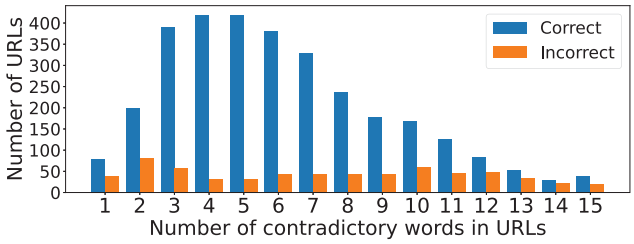


Fig. 2. Correctly and incorrectly classified ISCX test URLs with contradictory words, by URLTran trained on EBBU.

URLs containing new words and contradictory words were more often correctly classified than incorrectly classified. Fig. 2 illustrates one example of this observation. The figure displays the number of URLs from the ISCX test set that are correctly and incorrectly classified by the URLTran model trained on the EBBU dataset based on the number of contradictory words within the test URLs. We observe that the number of correctly classified URLs is generally greater than the number of incorrectly classified URLs.

To test the statistical significance of this observation, for different counts of new and contradictory words in each URL, we calculated the difference between the number of correctly classified and incorrectly classified URLs and conducted sign tests [60]. The sign test is a statistical test that checks for consistent differences between two samples of observations. That is, the sign test considers the median of the differences between paired observations and checks to statistical significance, whether it is either equal to, greater than or less than an expected value. Thus for our analysis, we apply this test to the values of difference between the number of correctly and incorrectly classified URLs.

As the sign test checks whether the median of a group of values (difference values between the number of correctly and incorrectly classified URLs) is equal to an expected value or not, it is similar to the one sample t-test. However, the t-test assumes a normal distribution for the sample, while our sample of difference values displayed a skewed distribution. Therefore, we selected the sign test, which is non-parametric and does not assume that the data follows any particular distribution.

For each training and test dataset combination, we performed three separate sign tests which tested the null hypothesis that the median difference value is equal to zero against three alternative hypotheses separately:

1. *Two-sided alternative:* the median difference value is not equal to zero
2. *Greater-than alternative:* the median difference value is greater than zero
3. *Less-than alternative:* the median difference value is less than zero

The p-value results of the sign tests are shown in Table 3.

We first discuss the results for URLNet and URLTran. For most tests with the two-sided and greater-than alternative, the p-value is statistically significant (less than 0.05). Conversely for all tests with the less-than alternative save for one, the p-values are insignificant (greater than 0.05). This reflects that although the null hypothesis (that the median difference is equal to 0) cannot be rejected in favour of alternative hypothesis 3, it can be rejected in favour of alternative hypotheses 1 and 2, indicating that the median difference between number of correctly classified URLs and incorrectly classified URLs is significantly greater than 0. In other words, of the URLs containing new or contradictory words, the number of correctly classified URLs are significantly greater than the number of incorrectly classified URLs. Thus we conclude that new or contradictory words are not a significant source of transfer error for URLNet and URLTran.

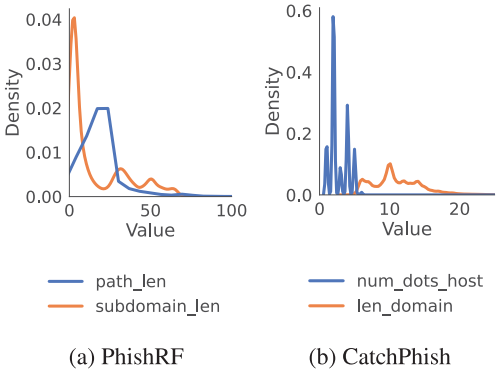


Fig. 3. Example feature distributions for PhishRF and CatchPhish.

We conducted the same analysis for the CatchPhish model with a slightly different approach with respect to the *new words* analysis. Unlike URLNet and URLTran, we did not consider the number of new words in the URL but instead the ratio of new words to the total number of words in the URL. This is because CatchPhish uses TF-IDF features using a vocabulary built from the training dataset only, and therefore, *new words* appearing in the test sets are not directly accounted for during prediction. However, we hypothesise that the proportion of new words in the URL may have an indirect effect on the model's predictability as a higher proportion of new words means less TF-IDF information available overall.

The p-value results are shown in Table 3. Unlike the URLNet and URLTran models, we observe that most of the p-values for all tests are insignificant, implying that regardless of the alternative hypothesis, the null hypothesis cannot be rejected. This indicates that for the CatchPhish model, new and contradictory words are a source of transfer error. One reason for this may be that CatchPhish only accounts for the frequency-based information for each word token (TF-IDF). Unlike URLNet and URLTran which uses word embeddings, it does not consider the contextual information related to each word in the input URL. As a result, it has lower generalisability and produces more incorrect predictions when new or contradictory words appear in test URLs.

We conclude from these results that distributional differences between the vocabulary of training and test datasets, in the form of new or contradictory words, do not adversely affect the transfer performance of URLNet and URLTran. It does however hamper the transfer performance of CatchPhish. Therefore for URLNet and URLTran, we do not conduct any vocabulary-based modifications in our experiments as outlined in Section 7. Instead, we focus on other relevant features such as path length.

6.2. Distributional differences in features

Next, we focus on the hand-crafted features used in CatchPhish and PhishRF, and explain how influential features were extracted for this analysis. As CatchPhish and PhishRF are random forest models, we use mean decrease in impurity [61] to first identify the top 15 most important features for each model, trained on each of the three training sets. Afterwards, for each model, we selected the features that were common across all datasets. For PhishRF this resulted in seven common top features and for CatchPhish this resulted in six common top features across the three datasets. Table 4 lists these top features for the two models. Next, we show that there are statistically significant distributional differences in some of these features among the three datasets that cause a drop in transfer accuracy.

To determine whether the distributional differences of these features between training and test sets are statistically significant, we

Table 3
Sign test p-values: Testing whether the median difference between correctly and incorrectly classified URLs containing new words or contradictory words is equal to 0. Significant p-values (less than 0.05) are shown in bold.

New words						Contradictory words					
			Alternative Hypotheses						Alternative Hypotheses		
Model	Train	Test	Two-sided	Greater	Less	Model	Train	Test	Two-sided	Greater	Less
URLTran	HP	EBBU	**3.91e−03**	**1.95e−03**	1.00e+00	URLTran	HP	EBBU	**3.55e−15**	**1.78e−15**	1.00e+00
		ISCX	**7.81e−03**	**3.91e−03**	1.00e+00			ISCX	**2.83e−06**	**1.41e−06**	1.00e+00
	EBBU	HP	6.25e−02	**3.13e−02**	1.00e+00		EBBU	HP	**6.62e−05**	**3.31e−05**	1.00e+00
		ISCX	2.19e−01	1.09e−01	9.84e−01			ISCX	**1.57e−04**	**7.83e−05**	1.00e+00
	ISCX	HP	1.25e−01	9.92e−01	6.25e−02		ISCX	HP	**5.10e−03**	**2.55e−03**	9.99e−01
		EBBU	6.54e−02	9.94e−01	**3.27e−02**			EBBU	**6.04e−07**	**3.02e−07**	1.00e+00
URLNet	HP	EBBU	**1.19e−07**	**5.96e−08**	1.00e+00	URLNet	HP	EBBU	**3.52e−02**	**1.76e−02**	9.96e−01
		ISCX	**1.83e−03**	**9.16e−04**	1.00e+00			ISCX	5.81e−01	8.67e−01	2.91e−01
	EBBU	HP	**4.34e−03**	**2.17e−03**	1.00e+00		EBBU	HP	**2.25e−02**	**1.12e−02**	9.98e−01
		ISCX	**3.13e−02**	**1.56e−02**	1.00e+00			ISCX	6.54e−02	**3.27e−02**	9.94e−01
	ISCX	HP	**1.54e−03**	**7.72e−04**	1.00e+00		ISCX	HP	3.02e−01	9.41e−01	1.51e−01
		EBBU	**1.69e−02**	**8.45e−03**	9.98e−01			EBBU	6.54e−02	**3.27e−02**	9.94e−01
CatchPhish	HP	EBBU	1.25e−01	6.25e−02	9.92e−01	CatchPhish	HP	EBBU	6.54e−02	**3.27e−02**	9.94e−01
		ISCX	7.03e−02	**3.52e−02**	9.96e−01			ISCX	1.00E+00	6.23e−01	6.23e−01
	EBBU	HP	2.89e−01	1.45e−01	9.65e−01		EBBU	HP	**3.91e−02**	**1.95e−02**	9.98e−01
		ISCX	**3.13e−02**	**1.56e−02**	1.00e+00			ISCX	1.09e−01	5.47e−02	9.89e−01
	ISCX	HP	2.89e−01	1.45e−01	9.65e−01		ISCX	HP	1.00E+00	5.00e−01	7.46e−01
		EBBU	1.18e−01	5.92e−02	9.82e−01			EBBU	3.44e−01	1.72e−01	9.45e−01

Table 4
Common important features of PhishRF and CatchPhish models trained on HP, EBBU, and ISCX.

PhishRF
1. alexa_domain: Is the domain in Alexa Top one million list 2. subdomain_len: Length of subdomain 3. keyword_count: Number of keywords in the URL 4. raw_word_count: Number of words obtained after parsing the URL by special characters 5. path_len: Length of path 6. std_word: Standard deviation of word lengths in the raw word list 7. avg_word_len: Average length of the words in the raw word list
CatchPhish
1. http: TF-IDF of the term http 2. num_dots_host: Number of dots in hostname 3. www: TF-IDF of the term www 4. len_domain: Length of the domain 5. avg_word_len_host: Average word length in the hostname 6. longest_word_len_host: Longest word length in the hostname

use the Mann–Whitney U test [62]. The Mann–Whitney U test checks whether two independently sampled groups come from the same distribution. Although the *two-sample t-test* [63] is a popular alternative for this test, it assumes that both samples follow a normal distribution. For our analysis, the two samples are the feature values from the training and test datasets, and an initial check showed that the features do not follow the normal distribution. The distribution of some features is illustrated in Fig. 3. Thus we selected the non-parametric Mann–Whitney U test, which does not require the samples to follow any particular distribution.

Since the Mann–Whitney U test is known to produce very low p-values when the sample size is very large [64], we randomly sample 100 data points for each feature from each dataset for reliable results. We also leave out the binary variable *alexa_domain* from our analysis since this test supports continuous variables only.

We show the resulting p-values in Table 5, for PhishRF and CatchPhish, respectively, where the insignificant p-values (> 0.05) are shown in bold. Insignificant p-values imply that the null hypothesis cannot be rejected — that the pair of train and test set feature values can be considered to have the same distribution. On the other hand, significant p-values indicate that the null hypothesis should be rejected in favour of the alternative hypothesis which is that the train and test feature values come from different distributions. Therefore, when the *p*-value for the train and test set pair of the same dataset is insignificant, but the p-values for the train and test pairs of different datasets are significant, it indicates that overfitting with respect to the given feature may contribute to low transfer performance.

In Table 5, for PhishRF, when the test sets come from the same dataset as the train sets, the p-values are insignificant across all common top features, indicating a similar distribution. In contrast, the p-values are mostly significant (< 0.05) when the test set is not from the same dataset as the training set. The exceptions to this pattern are the *std_word* and *keyword_count* features. This indicates that if PhishRF overfits with respect to the four other top features, it may cause poor transfer performance on the other datasets. The results for CatchPhish show a similar pattern except for the TF-IDF features for the term *http*.

Overall, for each model, we shortlist a few features which are likely to contribute to low transfer performance if models overfit on them. We shortlist by selecting the features that return insignificant p-values (> 0.05) when the test set is from the same dataset as the training set, and significant p-values when the test set is from a different dataset as the training set. For PhishRF these features are *path_len* and *raw_word_count*, and for CatchPhish these features are *num_dots_host* and *len_domain*. In Fig. 4 we illustrate our method of determining these overfitting features through the statistical tests, using the example of the CatchPhish model.

7. URL feature alignment

In the previous section, we identified the model-specific features that can hamper cross-dataset performance. Here, we modify the distributions of those features in the source dataset (i.e., training set) to improve the cross-dataset performance. We note that while some of the URL feature-related insights mentioned in this section may be well known, our main contribution in this paper is our approach to minimising the feature distribution discrepancies between the source and target domains to improve cross-dataset performance.

We outline the features specific to the different models and our alignment process for each of them below. For all models, we removed the 'http', 'https', and 'www' from the beginning of URLs as these are

Table 5
Mann–Whitney U Test P-values: Testing the null hypothesis that the distributions of the common important features are the same for different training and test set pairs. The insignificant p-values are shown in bold.

		Common top important features — PhishRF					
Train set	Test set	path_len	raw_word_count	subdomain_len	avg_word_len	std_word	keyword_count
HP	HP	**5.06e−01**	**8.93e−01**	**2.77e−01**	**8.98e−01**	**7.44e−01**	**2.43e−01**
	EBBU	5.51e−07	5.02e−05	1.10e−11	2.84e−03	**8.95e−01**	**1.74e−01**
	ISCX	3.28e−12	5.74e−18	4.83e−09	2.68e−04	**3.39e−01**	**9.67e−02**
EBBU	HP	5.30e−05	6.98e−03	4.56e−15	9.15e−03	**1.64e−01**	**3.77e−01**
	EBBU	**5.99e−01**	**5.54e−01**	**9.77e−01**	**1.68e−01**	**2.27e−01**	**4.31e−01**
	ISCX	3.24e−05	3.61e−09	**7.30e−01**	1.82e−02	**1.51e−01**	**5.36e−01**
ISCX	HP	6.93e−14	2.32e−13	2.84e−17	4.91e−03	1.89e−02	**2.57e−01**
	EBBU	3.09e−12	2.28e−11	1.19e−04	**6.96e−01**	3.87e−02	**6.64e−01**
	ISCX	**4.64e−01**	**1.38e−01**	**7.37e−01**	**2.88e−01**	**5.21e−01**	**8.61e−01**
		Common top important features — CatchPhish					
Train set	Test set	num_dots_host	len_domain	www	http	avg_word_len_host	longest_word_len_host
HP	HP	**6.17e−01**	**4.42e−01**	**3.15e−01**	2.57e−02	**5.71e−01**	**1.62e−01**
	EBBU	1.04e−06	6.85e−07	**1.06e−01**	2.38e−10	1.55e−03	6.85e−03
	ISCX	5.63e−11	4.54e−03	5.00e−07	9.52e−10	5.44e−04	5.31e−05
EBBU	HP	1.13e−08	4.11e−11	6.25e−05	**5.07e−01**	**1.34e−01**	**6.20e−01**
	EBBU	**9.32e−01**	**6.26e−01**	**6.93e−01**	**2.02e−01**	**1.22e−01**	**8.00e−01**
	ISCX	3.90e−04	1.66e−04	2.86e−06	6.11e−03	**1.02e−01**	4.85e−02
ISCX	HP	5.00e−12	2.58e−02	1.06e−06	9.53e−05	1.12e−03	3.51e−04
	EBBU	1.68e−05	7.17e−06	2.93e−02	**7.06e−02**	3.11e−02	1.11e−02
	ISCX	**5.32e−01**	**6.02e−01**	**5.23e−01**	**5.20e−01**	**9.72e−01**	**3.47e−01**

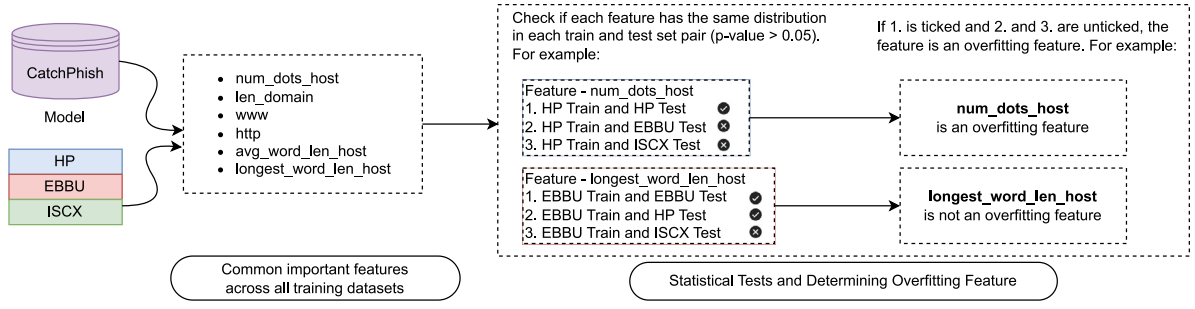


Fig. 4. Identifying overfitting features for CatchPhish through statistical tests.

not strong phishing or benign indicators and are likely to serve as noise. We refer to this modification as *removing the URL prefix* in the rest of the paper.

7.1. Changing path length

For PhishRF, we found *path_len* and *raw_word_count* to be an important feature with distributional differences among the three datasets. Since these two features are likely to be correlated, we only select the path length. Subsequently, we match the path length distributions of the source training sets to that of the target test set by either truncating or extending URL paths using a random selection of tokens from the path vocabulary of the training dataset.

The overall process we follow to match the path length is described in Algorithm 1 using benign URLs as an example. We first collect the set of words that form the path vocabulary of the source training dataset (V). We also separately collect the benign (b) and phishing URLs of the training dataset and sort them in decreasing order of path length. Next, we obtain the empirical distribution of target test set URL path lengths (S) and compare the frequency of test URLs with the frequency of benign URLs (m) from the training dataset. If the target dataset URL frequency ($|S|$) is larger than the benign training URL frequency (m), we take a random sample from the target set path length distribution. The size of this random sample matches the number of benign training URLs, and this becomes the desired path length distribution (P_T) for the benign training URLs. Otherwise, if the target URL frequency ($|S|$) is smaller than the benign training URL frequency (m), we upsample the target path length distribution (S) using nearest neighbour interpolation to a size that matches the number of the benign training URLs (m) and this becomes the desired path length frequency

Algorithm 1 Modifying the source EBBU benign training path length distribution to match the target ISCX test set path length distribution

V: Set of path vocabulary from the EBBU training set
b: List of all EBBU training benign URLs in reverse order of path length
S: Path length distribution of ISCX target test set
$m = |b|$
$P_T \leftarrow$ UPSAMPLE(S, m) ▷ Since $|S| <= m$
$Modified_URLs \leftarrow []$
for i in $0, 1, ...m$ **do**
  if $P_T[i] >=$ GETPATHLENGTH$(b[i])$ **then**
    $x = P_T[i] -$ GETPATHLENGTH$(b[i])$
    $Modified_URLs[i] \leftarrow$ APPENDPATH$(b[i], x, V)$
  else
    $y =$ GETPATHLENGTH$(b[i]) - P_T[i]$
    $Modified_URLs[i] \leftarrow$ TRUNCATEPATH$(b[i], y)$
  end if
end for
return $Modified_URLs$

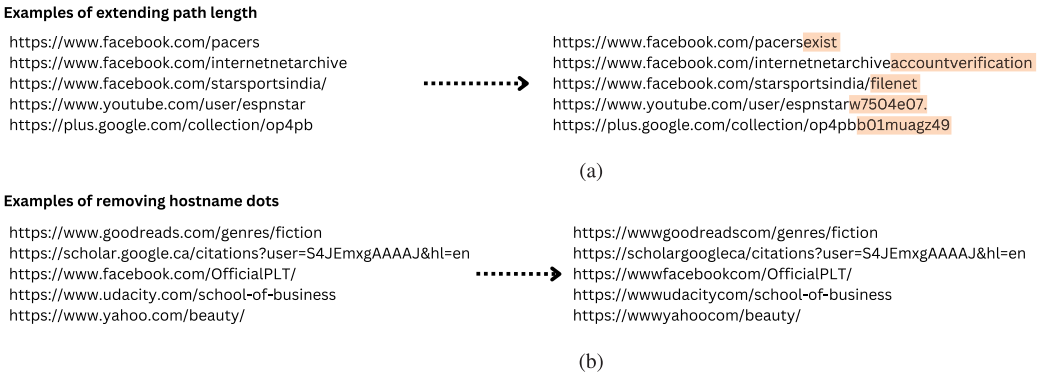


Fig. 5. Examples of feature modifications.

distribution (P_T) for the benign training URLs. In Algorithm 1 we refer to this upsampling approach as the UPSAMPLE(S, m) function.

We then iterate through (P_T) in reverse order of path length and simultaneously iterate through the set of benign training URLs (b). For each URL in (b), we calculate the original path length. We refer to this using the GETPATHLENGTH(URL) function in Algorithm 1 which counts the number of tokens in the URL path. We then compare the original path length of the URL to the desired path length from (P_T). If the desired path length is longer than the original path length we extend the path by appending randomly selected words from the path vocabulary of the training dataset (V). This is referred to as the APPENDPATH() function in Algorithm 1. If the desired path length is shorter than the original path length we truncate the path of the URL to the desired path length and this is referred to as the TRUNCATEPATH() function in Algorithm 1. In this way, we match the benign URL path length distribution to the target path length distribution. We conduct the same method for the phishing training URLs and finally obtain a whole training set with a path length distribution that matches the target path length distribution.

Note that our approach does not involve labelled path length information from the target test set. This makes our approach fall under Unsupervised Domain Adaptation. Furthermore, unlike modifying the domain of a URL, modifying the path length in this way does not affect the phishing-relevant information held by the words in a URL. Generally, the domain contains more phishing-relevant information within the hostname and top-level domain, which is why traditional phishing detection models use multiple domain-based word-level features. Examples include checking whether the hostname exists in the Alexa top domains list or not. In contrast, the actual words in the URL path contribute less to the maliciousness of a URL. Therefore adding or removing words from the URL path does not affect the phishing-relevant discriminatory information in the URL.

In Fig. 5(a), we show some example data points from the EBBU training set for which the path length was extended to match that of the ISCX test set path length distribution, and Fig. 6 illustrates how the path length distributions look like once we do the alignment.

Finally, for URLNet and URLTran, the input features are mainly the tokens and the sequence length. As we concluded in Section 6.1, new and contradictory words are not the source of error, for these two models, we also focus on modifying the path length as it is a sequence length feature with distributional differences across different datasets. Thus for those two models, we match the distributions in the same way as we do for PhishRF.

7.2. Removing hostname dots

For CatchPhish, we found the *num_dots_host* (number of dots in the hostname) and *domain_len* (domain length) to be important features with distributional differences among the three datasets. Unlike our approach with path length distribution, we decided against modifying the domain length by altering domain tokens. This is because the domain generally contains strong benign/phishing indicators, such as whether the domain exists in the Alexa top domains list. Therefore, for CatchPhish, we focus mainly on the number of dots in the hostname and modify this by removing all dots from the hostname in the source and target datasets. This is exemplified in Fig. 5(b).

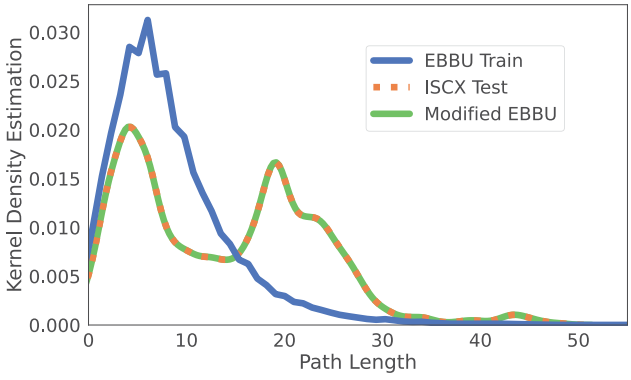


Fig. 6. Change in EBBU training path length distribution after modifying to match ISCX test set path length distribution.

*To summarise, for URLNet, URLTran and PhishRF we **change the path length** and for CatchPhish we **remove the hostname dots**, and for all models we further experiment with **removing the URL prefix**.*

8. Results

We next discuss the results of applying the feature modifications explained in the previous section. In Table 6, we report the F1 scores for the model-specific modifications that returned the best results. That is, we report the combinations of feature modifications that returned the best improvement in F1 scores. As mentioned in Section 5, we report the F1 scores as it provides a balanced reflection of the true positive and false positive rates. Overall, out of the 24 scenarios we tested, 19 scenarios resulted in a performance improvement.

8.1. URLNet

For URLNet, the best improvements in F1 score were found to be when training on HP and testing on ISCX and vice versa. When trained on HP and evaluated on ISCX, the F1 score improved from 0.72 to 0.85 which corresponds to a total accuracy improvement from 66% to 84%. When trained on ISCX and tested on HP, the F1 score improved from 0.68 to 0.71 which corresponds to a total accuracy improvement

Table 6
Transfer performance: F1 scores before and after applying feature modifications.

	Train	Test	Before	After	Diff	Modification
URLNet	ISCX	HP	0.68	0.71	0.03	remove prefix+change path
	ISCX	EBBU	0.69	0.74	0.05	remove prefix+change path
	EBBU	HP	0.92	0.88	−0.04	change path
	EBBU	ISCX	0.88	0.88	0.00	change path
	HP	EBBU	0.79	0.86	0.07	change path
	HP	ISCX	0.72	0.85	**0.13**	remove prefix+change path
Average Increase					0.04	
URLTran	ISCX	HP	0.67	0.78	**0.11**	remove prefix+change path
	ISCX	EBBU	0.68	0.77	0.09	remove prefix+change path
	EBBU	HP	0.93	0.93	0.00	change path
	EBBU	ISCX	0.83	0.91	0.08	remove prefix+change path
	HP	EBBU	0.86	0.89	0.03	remove prefix+change path
	HP	ISCX	0.75	0.79	0.04	remove prefix+change path
Average Increase					0.06	
PhishRF	ISCX	HP	0.67	0.83	**0.16**	remove prefix+change path
	ISCX	EBBU	0.68	0.87	**0.19**	remove prefix+change path
	EBBU	HP	0.94	0.95	0.01	change path
	EBBU	ISCX	0.87	0.87	0.00	change path
	HP	EBBU	0.86	0.91	0.05	change path
	HP	ISCX	0.83	0.84	0.01	remove prefix+change path
Average Increase					0.07	
CatchPhish	ISCX	HP	0.68	0.80	**0.12**	remove hostname dots
	ISCX	EBBU	0.68	0.75	0.07	remove prefix + hostname dots
	EBBU	HP	0.88	0.90	0.02	remove prefix + hostname dots
	EBBU	ISCX	0.80	0.89	0.09	remove hostname dots
	HP	EBBU	0.72	0.69	−0.03	remove prefix + hostname dots
	HP	ISCX	0.73	0.76	0.03	remove hostname dots
Average Increase					0.05	
Overall Average Increase					0.06	

from 52% to 59%. Between the two modification types, the average increase in F1 score for URLNet was 0.04 and the average increase in total accuracy was 6%.

Only when training URLNet on EBBU, there was no improvement in F1 score. That is, after the path length modification, evaluating on HP returned a decrease in F1 score and when evaluating on ISCX, the F1 score remained the same. Further investigation showed that this was caused by a limitation of the Unsupervised Domain Adaptation approach which is that the approach is agnostic to the class-based feature distribution of the target domain. As seen in Fig. 7, the original path length distribution of EBBU was such that both its benign and phishing boxplot ranges overlaid the benign and phishing ranges of HP. However, when modifying the path length of EBBU to match that of HP, the Unsupervised Domain Adaptation approach requires that the class-based HP distributions be ignored and instead that the overall EBBU path length distribution be matched to the overall HP path length distribution. As a result, it is possible for some of the original class-specific similarities between the source and target domain to be lost after the feature modification.

In Fig. 8, we illustrate that this was the case for the EBBU training set when its path length was modified to match that of HP. With respect to matching the EBBU distribution to ISCX, Fig. 8 shows that after the path length modification, the EBBU benign distribution was still not as similar to the ISCX benign distribution, even though the overall distribution was matched (i.e., Fig. 6).

8.2. URLTran

For URLTran, the two greatest improvements in F1 score were found to be when training the model on ISCX. Here the F1 score increased from 0.67 to 0.78 when evaluated on HP, which corresponds to a total accuracy improvement from 52% to 72%. When we evaluated on EBBU, the F1 score improved from 0.68 to 0.77 which corresponds to a total accuracy improvement from 56% to 70%. Overall, the average increase in F1 score for URLTran was 0.06 and the average increase in total accuracy was 7%, indicating the effectiveness of our approach.

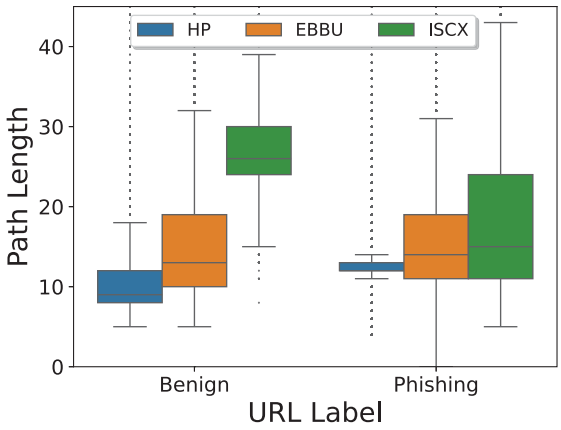


Fig. 7. Boxplot of path length distributions of the HP, EBBU and ISCX datasets.

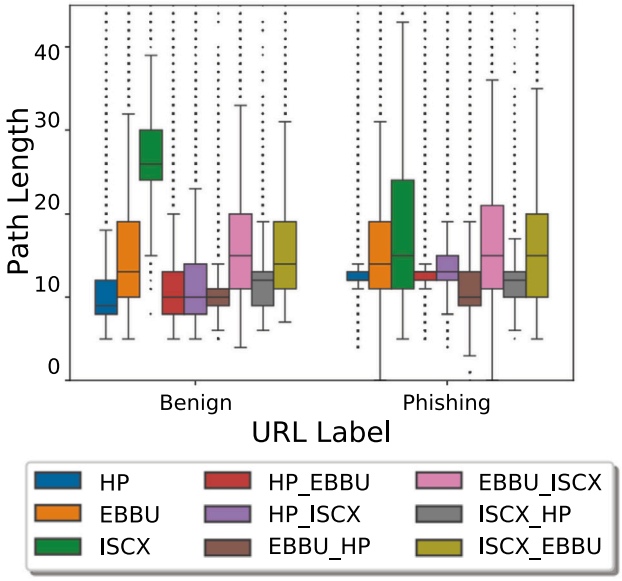


Fig. 8. Boxplot of path length distributions. For distributions where the path length has been modified, the training set name is followed by the test set name. For example, HP_EBBU refers to the HP training set modified to match the EBBU test set.

8.3. PhishRF

For PhishRF, the highest improvement in F1 score was obtained when PhishRF was trained on ISCX with the feature modifications. For this case when we evaluated on HP, the F1 score increased from 0.67 to 0.83 and the total accuracy improved from 51% to 63%. When we evaluated on EBBU, the F1 score improved from 0.68 to 0.87 which corresponds to a total accuracy improvement from 53% to 80%. Overall the modifications resulted in an average F1 score improvement of 0.07 and an average improvement in total accuracy by 7%.

For the case of training PhishRF on EBBU and testing on ISCX, the F1 score remained the same after the feature modifications. One possible reason for this may be that since the classification model is a random forest, the top important features of the model can change due to a distribution change in a feature that was originally important. Therefore, changing the path length of EBBU to match that of ISCX may have caused some other features to become important during training

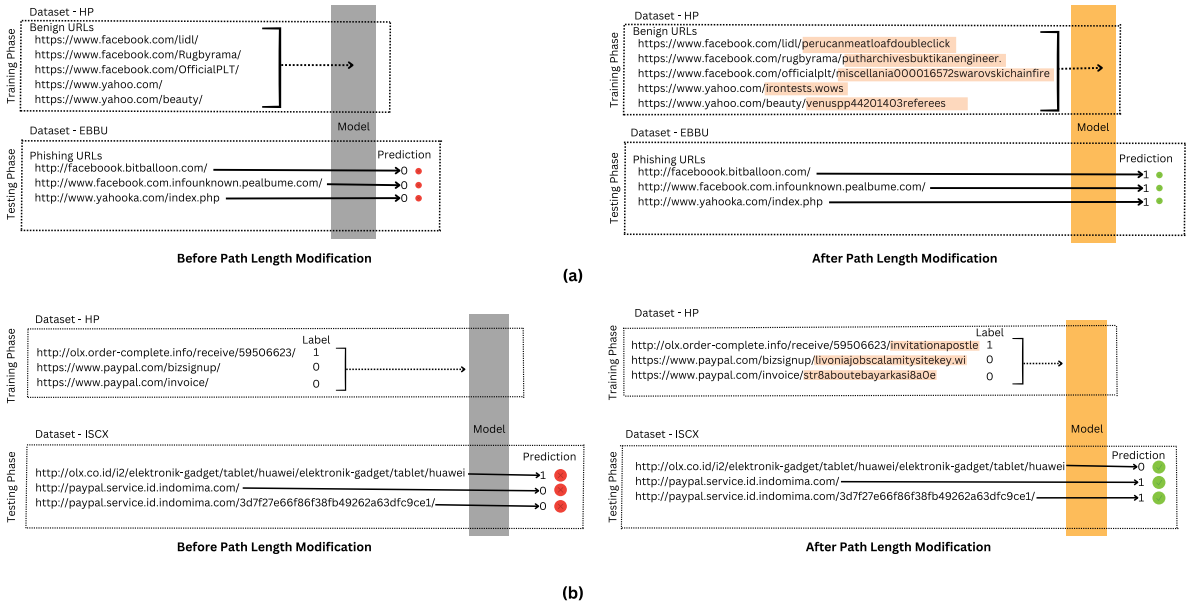


Fig. 9. Examples of improvement in phishing detection after feature modifications.

that caused lower transfer performance or at least kept the F1 score the same.

8.4. CatchPhish

For CatchPhish, the highest improvement in F1 score was also obtained when CatchPhish was trained on ISCX with the feature modifications. For this case when we evaluated on HP, the F1 score increased from 0.68 to 0.80 and the total accuracy improved from 58% to 76%. When we evaluated on EBBU, the F1 score improved from 0.68 to 0.75 and the total accuracy improved from 56% to 67%. Over all cases, the average increase in F1 score was 0.05 and the average increase in total accuracy was 7%.

Only when training on HP and testing on EBBU, the modifications did not improve the F1 score. It is important to note that as the classification model is a random forest model, by removing all the dots from the hostname we essentially remove the feature *hostname* from the model altogether. As a result, there is a rearrangement of the importance assigned to the remaining features during retraining and the resultant important features may adversely affect transfer performance. To investigate this, we observed the top 15 features before applying any feature modification, after removing the hostname dots, and after removing the prefix in addition to removing the hostname dots. We found that before applying any modifications, the top 15 important features included 7 TF-IDF features out of which only two were for words not common in general URLs: 'uwbrvz8quj' and 'page1'. The five remaining words were 'https', 'php', 'http', 'com', and 'www'. After removing the hostname dots, we found that the top 15 important features again included 7 TF-IDF features, however, none of the words were common URL terms. Some examples include 'limetorrents' and 'fundraiser'. After removing the prefix and hostname dots, 11 out of the 15 top features were TF-IDF features and again, none of the words were common URL terms. When the top features depend on terms that are not likely to be found in the test datasets, the transfer performance is naturally likely to decrease.

Across all models and all training and test set pairs, we conclude that the modifications result in an average F1 score improvement of 0.06 and an average accuracy improvement of 7%.

8.5. Examples for performance improvement

Here, we present a few examples of how changing path length returned performance improvement on actual phishing URLs. Originally, the median benign path length in the HP dataset was lower than its median phishing path length (**cf**. Fig. 7). Therefore, models trained on HP would learn to classify shorter URLs as benign, and when evaluated on EBBU, some short phishing URLs were misclassified as benign. However, modifying the path length distribution in HP to match that of EBBU removed some of the class-based bias within the HP training URLs and allowed previously misclassified EBBU URLs to be correctly classified. We illustrate this effect in Fig. 9(a) through some examples. On the top part of the figure, we give examples of some benign URLs in the HP dataset that were extended in path length and on the bottom, we exemplify how this modification resulted in correct predictions at test time.

In Fig. 9(b) we provide similar examples but this time when testing on the ISCX dataset. We show some examples of URLs with similar domain names but opposite labels in the HP training and the ISCX test set. Before the path length modification the URLTran models incorrectly predicted the ISCX test URLs to have the same label as the URL with a similar domain name in the HP train set. However after the path length modification, the model returned the correct predictions for the ISCX test URLs.

9. Discussion

Overall, the unsupervised framework we proposed improved the cross-dataset performance of URL classification models. It is important to note that it would not be a fair comparison to compare the cross-dataset performance presented in the previous section with the accuracies presented in existing work involving phishing URL models. This is because while other works mainly propose new models and evaluate them on test sets from the same source domain as the training sets, our main focus was to propose a UDA-based approach to improve the performance of existing models across different target domain test sets.

Due to the fact that our approach is unsupervised, it does not require any effort in manual annotation when building models on target datasets. This can be specifically useful in the area of threat intelligence sharing. As different companies collect URL data in different ways,

our framework allows leveraging these datasets in aggregation to build better-performing models. We next discuss the implications of our results, limitations and possible future research directions.

9.1. Feature differences in URL datasets

Our analysis showed that the low transfer performance of existing URL classifiers is primarily due to variations in path length and URL prefixes across datasets. Although URLs typically follow a standard format, these discrepancies likely arise from different data collection methods.

For example, with regards to the prefix, we observed that benign URLs in the HP dataset mostly began with https and phishing URLs mostly began with http while most benign URLs in EBBU began with http. Models trained on the HP dataset, therefore, would misclassify some benign URLs in EBBU starting with http as phishing. One possible reason for such a discrepancy is that some of the benign URLs in the HP dataset were collected by querying the Google Search Engine API with the Alexa top domains, whereas the benign URLs in EBBU were collected using the Yandex Search Engine API. As the Google Search Engine API is known to prioritise HTTPS URLs in search results [65], this may have contributed to the larger proportion of HTTPS benign URLs in the HP dataset in comparison to the EBBU dataset. As discrepancies in the URL prefix are non-indicative of the maliciousness of a URL, they lead to overfitting during training and, therefore, low cross-dataset performance.

Similarly, the path length is another feature affected by how the URLs are sourced. For example, when collecting HP benign URLs, the location for the Google search engine queries was set to the United States. The search results that were collected were only those that returned URLs for web pages in the English language and files such as PDF and Word documents were filtered out. Our analysis showed that this process generally resulted in shorter HP benign URL path lengths than those in the ISCX dataset (**cf.** Fig. 7). The ISCX dataset obtained benign URLs using the Heitrix web crawler with the Alexa top domains as the seed input which returned URLs with longer path lengths. A possible reason for this is that, unlike search engines that prioritise returning relevant results efficiently, web crawlers conduct a more comprehensive search through web content and, therefore, are more likely to return longer and more detailed subdirectories in the URL path than search engines. Therefore, a model trained on the ISCX dataset would expect benign URLs to have longer paths and when it is evaluated on the HP dataset, it may misclassify benign URLs with very short path lengths as phishing.

Based on these observations, we highlight that when forming a training dataset for standalone phishing URL classification, it is important to ensure that the path length distribution, in particular, is representative of the general class-based distribution. As the prefixes are generally non-indicative of the maliciousness of a URL, those can be removed during pre-processing. Also, we recommend that when releasing URL datasets, it is beneficial to explain in detail the data collection method and any pre-processing methods that have already been applied.

9.2. Limitations and future work

Our analysis and approach investigated the transferability of individual features separately. While this revealed specific dataset-related issues that impede transfer performance specifically in phishing detection models, it did not account for inter-correlations among multiple features. The effect of this limitation is observed for CatchPhish in particular where modifying one feature (number of hostname dots) caused the model to overfit on other non-relevant features (**cf.** Section 8.4). This problem could be addressed by integrating correlated-based feature selection methods [66] into the feature alignment framework.

Another limitation relates to the nature of Unsupervised Domain Adaptation which assumes that the target domain is unlabelled. As a result, when using the target domain distribution to modify our source domains, we only matched the overall feature distributions and not the class-specific distributions. We saw in the case of URLNet, that when modifying EBBU path lengths to match that of HP, the original similarity in benign path length distribution between the two datasets was lost and this decreased the transfer performance (**cf.** Section 8.1). Incorporating a limited amount of labelled target data in conjunction with our method may be a possible solution to this issue.

We also observed that between URLNet and URLTran, URLTran responded better to the feature modifications as it produced a higher average increase in F1 score even though the same feature modifications were applied to both. As URLTran is a transformer-based model, it is better able to capture the contextual information of the URL words/tokens during its pretraining stage using masked language modelling. As it is also trained on top of the pre-trained BERT model, which was trained on an enormous amount of text data from the internet, it is also likely to contain contextual information about benign URLs. We believe this is why it can overcome the class-specific differences that arose post feature modifications, which resulted in a decrease in F1 score for one URLNet scenario. As such, it is possible that further performance improvements can be achieved by using Large Language Models (LLMs) such as GPT-3, GPT-4, and LLaMA models [67–69], that are trained on much larger corpora of internet data and have more model parameters than BERT models.

10. Conclusion

In this work, we propose a framework for improving the generalisability of phishing URL classifiers based on Unsupervised Domain Adaptation. We demonstrate that state-of-the-art classifiers perform poorly when URLs come from sources different to the classifiers' original training sets and conduct a statistical analysis to find the specific features with distribution shifts between source and target datasets. Based on these findings, we propose an Unsupervised Domain Adaptation-based framework to align the features across the source and target URL datasets. Our results showed that the URL path length and prefixes are the primary factors causing low transfer performance across the source and target datasets. Our proposed feature-alignment framework resulted in an average F1 score increase of 0.06 and for some scenarios the increase was as high as 0.2. Overall, our work enables facilitating threat intelligence sharing between companies and allows researchers to combine multiple URL datasets obtained from different sources to build better models.

CRediT authorship contribution statement

Fariza Rashid: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Writing – original draft, Writing – review & editing. **Ben Doyle:** Conceptualization, Funding acquisition, Supervision. **Soyeon Caren Han:** Supervision. **Suranga Seneviratne:** Conceptualization, Funding acquisition, Project administration, Supervision.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Suranga Seneviratne reports financial support was provided by NSW Defence Innovation Network. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work was conducted in partnership with Thales Australia, funded by the NSW Defence Innovation Network (DIN) Postgraduate Research Scholarship Program.

References

[1] C.N. Kaspersky, Best bite: Kaspersky reveals phishing emails that employees find most confusing, 2022, https://www.kaspersky.com/about/press-releases/2022_best-bite-kaspersky-reveals-phishing-emails-that-employees-find-most-confusing. (Online; posted 22 June 2022).
[2] Google Inc, a. Google safe browsing, 2023, https://safebrowsing.google.com/. (Accessed 29 July 2023).
[3] S. Peters, Microsoft edge security for your business, 2023, https://learn.microsoft.com/en-us/deployedge/ms-edge-security-for-business. (Accessed 29 July 2023).
[4] A. Oest, Y. Safaei, P. Zhang, B. Wardman, K. Tyers, Y. Shoshitaishvili, A. Doupé, {Phishtime}: Continuous longitudinal measurement of the effectiveness of anti-phishing blacklists, in: 29th USENIX Security Symposium, USENIX Security 20, 2020, pp. 379–396.
[5] D. Canali, M. Cova, G. Vigna, C. Kruegel, Prophiler: a fast filter for the large-scale detection of malicious web pages, in: Proceedings of the 20th International Conference on World Wide Web, 2011, pp. 197–206.
[6] A.K. Jain, B.B. Gupta, A machine learning based approach for phishing detection using hyperlinks information, J. Ambient Intell. Humaniz. Comput. 10 (2019) 2015–2028.
[7] P. Zhang, A. Oest, H. Cho, Z. Sun, R. Johnson, B. Wardman, S. Sarker, A. Kapravelos, T. Bao, R. Wang, et al., Crawlphish: Large-scale analysis of client-side cloaking techniques in phishing, in: 2021 IEEE Symposium on Security and Privacy, SP, IEEE, 2021, pp. 1109–1124.
[8] M.S.I. Mamun, M.A. Rathore, A.H. Lashkari, N. Stakhanova, A.A. Ghorbani, Detecting malicious urls using lexical analysis, in: Network and System Security: 10th International Conference, NSS 2016, Proceedings, vol. 10, Springer, 2016, pp. 467–482.
[9] M. Sameen, K. Han, S.O. Hwang, Phishhaven—an efficient real-time ai phishing urls detection system, IEEE Access 8 (2020) 83425–83443.
[10] O.K. Sahingoz, E. Buber, O. Demir, B. Diri, Machine learning based phishing detection from urls, Expert Syst. Appl. 117 (2019) 345–357.
[11] R.S. Rao, T. Vaishnavi, A.R. Pais, Catchphish: detection of phishing websites by inspecting urls, J. Ambient Intell. Humaniz. Comput. 11 (2020) 813–825.
[12] R.S. Rao, T. Vaishnavi, A.R. Pais, Phishdump: A multi-model ensemble based technique for the detection of phishing sites in mobile devices, Pervasive Mob. Comput. 60 (2019) 101084.
[13] H. Le, Q. Pham, D. Sahoo, S.C. Hoi, Urlnet: Learning a url representation with deep learning for malicious url detection, 2018, arXiv preprint arXiv:1802.03162.
[14] D. Sahoo, C. Liu, S.C. Hoi, Malicious url detection using machine learning: A survey, 2017, arXiv preprint arXiv:1701.07179.
[15] J. Ma, L.K. Saul, S. Savage, G.M. Voelker, Beyond blacklists: learning to detect malicious web sites from suspicious urls, in: Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2009, pp. 1245–1254.
[16] P. Prakash, M. Kumar, R.R. Kompella, M. Gupta, Phishnet: predictive blacklisting to detect phishing attacks, in: 2010 Proceedings IEEE INFOCOM, IEEE, 2010, pp. 1–5.
[17] J. Ma, L.K. Saul, S. Savage, G.M. Voelker, Identifying suspicious urls: an application of large-scale online learning, in: Proceedings of the 26th annual international conference on machine learning, 2009, pp. 681–688.
[18] D.K. McGrath, M. Gupta, Behind phishing: An examination of phisher modi operandi, in: USENIX Workshop on Large-Scale Exploits and Emergent Threats, 2008, URL: https://api.semanticscholar.org/CorpusID:2069430.
[19] Y. Mourtaji, M. Bouhorma, D. Alghazzawi, G. Aldabbagh, A. Alghamdi, Hybrid rule-based solution for phishing url detection using convolutional neural network, Wirel. Commun. Mob. Comput. 2021 (2021) 1–24.
[20] G. Xiang, J. Hong, C.P. Rose, L. Cranor, Cantina+ a feature-rich machine learning framework for detecting phishing web sites, ACM Trans. Inf. Syst. Secur. 14 (2011) 1–28.
[21] K.L. Chiew, C.L. Tan, K. Wong, K.S. Yong, W.K. Tiong, A new hybrid ensemble feature selection framework for machine learning-based phishing detection system, Inform. Sci. 484 (2019) 153–166.
[22] T. Chin, K. Xiong, C. Hu, Phishlimiter: A phishing detection and mitigation approach using software-defined networking, IEEE Access 6 (2018) 42516–42531.
[23] W. Zhang, Q. Jiang, L. Chen, C. Li, Two-stage elm for phishing web pages detection using hybrid features, World Wide Web 20 (2017) 797–813.
[24] Y. Ding, N. Luktarhan, K. Li, W. Slamu, A keyword-based combination approach for detecting phishing webpages, Comput. Secur. 84 (2019) 256–275.
[25] Y. Lin, R. Liu, D.M. Divakaran, J.Y. Ng, Q.Z. Chan, Y. Lu, Y. Si, F. Zhang, J.S. Dong, Phishpedia: A hybrid deep learning based approach to visually identify phishing webpages, in: USENIX Security Symposium, 2021, pp. 3793–3810.
[26] X. Liu, J. Fu, Spwalk: similar property oriented feature learning for phishing detection, IEEE Access 8 (2020) 87031–87045.
[27] R.S. Rao, A.R. Pais, P. Anand, A heuristic technique to detect phishing websites using twsvm classifier, Neural Comput. Appl. 33 (2021) 5733–5752.
[28] C.L. Tan, K.L. Chiew, K.S. Yong, J. Abdullah, Y. Sebastian, et al., A graph-theoretic approach for the detection of phishing webpages, Comput. Secur. 95 (2020) 101793.
[29] G.G. Geng, X.D. Lee, Y.M. Zhang, Combating phishing attacks via brand identity and authorization features, Secur. Commun. Netw. 8 (2015) 888–898.
[30] R.S. Rao, A.R. Pais, Detection of phishing websites using an efficient feature-based machine learning framework, Neural Comput. Appl. 31 (2019) 3851–3873.
[31] M.M. Alani, H. Tawfik, Phishnot: a cloud-based machine-learning approach to phishing url detection, Comput. Netw. 218 (2022) 109407.
[32] P. Maneriker, J.W. Stokes, E.G. Lazo, D. Carutasu, F. Tajaddodianfar, A. Gururajan, Urltran: Improving phishing url detection using transformers, in: MILCOM 2021-2021 IEEE Military Communications Conference, MILCOM, IEEE, 2021, pp. 197–204.
[33] H. Yuan, Z. Yang, X. Chen, Y. Li, W. Liu, Url2vec: Url modeling with character embeddings for fast and accurate phishing website detection, in: 2018 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications, ISPA/IUCC/BDCloud/SocialCom/SustainCom, IEEE, 2018, pp. 265–272.
[34] C. McCormick, Word2vec tutorial-the skip-gram model, 2016, Apr-2016.[Online]. Available: http://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model.
[35] A. Aljofey, Q. Jiang, Q. Qu, M. Huang, J.P. Niyigena, An effective phishing detection model based on character level convolutional neural network from url, Electronics 9 (1514) (2020).
[36] J.D.M.W.C. Kenton, L.K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of NaacL-HLT, 2019, p. 2.
[37] G. Wilson, D.J. Cook, A survey of unsupervised deep domain adaptation, ACM Trans. Intell. Syst. Technol. 11 (2020) 1–46.
[38] J. Zhang, W. Li, P. Ogunbona, Joint geometrical and statistical alignment for visual domain adaptation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 1859–1867.
[39] D.A. Huang, Y.C.F. Wang, Coupled dictionary and feature space learning with applications to cross-domain image synthesis and recognition, in: Proceedings of the IEEE International Conference on Computer Vision, 2013, pp. 2496–2503.
[40] S. Gururangan, A. Marasović, S. Swayamdipta, K. Lo, I. Beltagy, D. Downey, N.A. Smith, Don't stop pretraining: Adapt language models to domains and tasks, 2020, arXiv preprint: 2004.10964.
[41] A. Ramponi, B. Plank, Neural unsupervised domain adaptation in nlp—a survey, 2020, arXiv preprint arXiv:2006.00632.
[42] S. Sun, B. Zhang, L. Xie, Y. Zhang, An unsupervised deep domain adaptation approach for robust speech recognition, Neurocomputing 257 (2017) 79–87.
[43] W.N. Hsu, Y. Zhang, J. Glass, Unsupervised domain adaptation for robust speech recognition via variational autoencoder-based data augmentation, in: 2017 IEEE Automatic Speech Recognition and Understanding Workshop, ASRU, IEEE, 2017, pp. 16–23.
[44] C. Wang, M. Niepert, H. Li, Recsys-dan: discriminative adversarial networks for cross-domain recommender systems, IEEE Trans. Neural Netw. Learn. Syst. 31 (2019) 2731–2740.
[45] M.A.A.H. Khan, N. Roy, A. Misra, Scaling human activity recognition via deep learning-based domain adaptation, in: 2018 IEEE International Conference on Pervasive Computing and Communications, PerCom, IEEE, 2018, pp. 1–9.
[46] S.A. Rokni, H. Ghasemzadeh, Synchronous dynamic view learning: a framework for autonomous training of activity recognition models using wearable sensors, in: Proceedings of the 16th ACM/IEEE International Conference on Information Processing in Sensor Networks, 2017, pp. 79–90.
[47] B. Gong, K. Grauman, F. Sha, Connecting the dots with landmarks: Discriminatively learning domain-invariant features for unsupervised domain adaptation, in: International Conference on Machine Learning, PMLR, 2013, pp. 222–230.
[48] J. Li, K. Lu, Z. Huang, L. Zhu, H.T. Shen, Transfer independently together: A generalized framework for domain adaptation, IEEE Trans. Cybernet. 49 (2018) 2144–2155.
[49] W. Xia, J. Huang, J.H. Hansen, Cross-lingual text-independent speaker verification using unsupervised adversarial discriminative domain adaptation, in: ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP, IEEE, 2019, pp. 5816–5820.
[50] A. Farahani, S. Voghoei, K. Rasheed, H.R. Arabnia, A brief review of domain adaptation, in: Advances in Data Science and Information Engineering: Proceedings from ICDATA 2020 and IKE, Vol. 2020, 2021, pp. 877–894.
[51] M. Sugiyama, M. Krauledat, K.R. Müller, Covariate shift adaptation by importance weighted cross validation, J. Mach. Learn. Res. 8 (2007).
[52] S.J. Pan, I.W. Tsang, J.T. Kwok, Q. Yang, Domain adaptation via transfer component analysis, IEEE Trans. Neural Netw. 22 (2010) 199–210.

[53] Y. Taigman, A. Polyak, L. Wolf, Unsupervised cross-domain image generation, 2016, arXiv preprint arXiv:1611.02200.
[54] E. Tzeng, J. Hoffman, T. Darrell, K. Saenko, Simultaneous deep transfer across domains and tasks, in: Proceedings of the IEEE International Conference on Computer Vision, 2015, pp. 4068–4076.
[55] Y. Ganin, V. Lempitsky, Unsupervised domain adaptation by backpropagation, in: International Conference on Machine Learning, PMLR, 2015, pp. 1180–1189.
[56] W. Aqeel, B. Chandrasekaran, A. Feldmann, B.M. Maggs, On landing and internal web pages: The strange case of jekyll and hyde in web performance measurement, in: Proceedings of the ACM Internet Measurement Conference, 2020, pp. 680–695.
[57] V. Drury, L. Lux, U. Meyer, Dating phish: An analysis of the life cycles of phishing attacks and campaigns, in: Proceedings of the 17th International Conference on Availability, Reliability and Security, 2022, pp. 1–11.
[58] A. Drichel, V. Drury, J. von Brandt, U. Meyer, Finding phish in a haystack: A pipeline for phishing classification on certificate transparency logs, in: Proceedings of the 16th International Conference on Availability, Reliability and Security, 2021, pp. 1–12.
[59] B. Van Dooremaal, P. Burda, L. Allodi, N. Zannone, Combining text and visual features to improve the identification of cloned webpages for early phishing detection, in: Proceedings of the 16th International Conference on Availability, Reliability and Security, 2021, pp. 1–10.
[60] W.J. Dixon, A.M. Mood, The statistical sign test, J. Amer. Statist. Assoc. 41 (1946) 557–566.
[61] L. Breiman, Random forests, Mach. Learn. 45 (2001) 5–32.
[62] H.B. Mann, D.R. Whitney, On a test of whether one of two random variables is stochastically larger than the other, Ann. Math. Stat. 5 (1947) 0–60.
[63] N. Cressie, H. Whitford, How to use the two sample t-test, Biom. J. 28 (1986) 131–148.
[64] M.W. Fagerland, L. Sandvik, The Wilcoxon–Mann–Whitney test under scrutiny, Stat. Med. 28 (2009) 1487–1497.
[65] Google Inc, b. Indexing HTTPS pages by default, 2023, https://developers.google.com/search/blog/{2015}/{12}/indexing-https-pages-by-default. (Accessed 12 October 2023).
[66] M.A. Hall, Correlation-based feature selection of discrete and numeric class machine learning, 2000.
[67] T. Brown, B. Mann, N. Ryder, M. Subbiah, J.D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, Adv. Neural Inf. Process. Syst. 33 (2020) 1877–1901.
[68] OpenAI, Gpt-4 technical report, 2023, arXiv:2303.08774.
[69] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, G. Lample, Llama: Open and efficient foundation language models, 2023, arXiv:2302.13971.



Fariza Rashid is a doctoral student at the University of Sydney, conducting research on applications of AI in cyber threat intelligence in collaboration with Thales Australia. She completed a Bachelor of Science and Bachelor of Advanced Studies (Hons) in Data Science and Financial Mathematics and Statistics from the University of Sydney in 2021. Her current work involves using deep learning, specifically sequential classification techniques, to identify benign indicators in malicious URL detection with a focus on reducing false positive rates.



Ben Doyle is an accomplished information security executive with over 20 years of international information security experience. Due to his many years of providing strategic C-level/board advice, he is able to apply technical threats into appropriate business contexts to determine overall risk to organisations. During this time Ben has been required to manage complex environments that requires balancing local and global organisational policies, domestic regulations, national security requirements, export control and international trade control restrictions, and applying the process, people and technology to ensure successful compliance. Ben maintains his strong passion for the technical aspects of information security and recently has been focusing on creating new business opportunities by working across the breath of the global organisation's capabilities while teaming and supporting small innovative start up technology companies.



Soyeon Caren Han is currently co-leader of AD-NLP (Australia Deep Learning NLP Group) and a senior lecturer (Associate Professor in U.S. System) at the University of Western Australia and an honorary senior lecturer (honorary Associate Professor in U.S. System) at the University of Sydney and the University of Edinburgh. AXer her Ph.D. (in 2017), she worked for 6 years at the University of Sydney. Her research interests include Natural Language Processing with Deep Learning. She is broadly interested in several research topics, including visual-linguistic multi-modal learning, abusive language detection, document layout analysis, and recommender system.



Suranga Seneviratne is a Senior Lecturer in Security at the School of Computer Science, The University of Sydney. He received his Ph.D. from University of New South Wales, Australia in 201ti. His current research interests include privacy and security in mobile systems, AI applications in security, and behaviour biometrics. Before moving into research, he worked nearly six years in the telecommunications industry in core network planning and operations. He received his bachelor degree from University of Moratuwa, Sri Lanka in 2005.